

def Jaccard-similarity(c):

$\Rightarrow \frac{\text{intersection}[A\text{-array}, B\text{-array}]}{\text{union}[A\text{-array}, B\text{-array}]}$

$\Rightarrow$ two documents are near duplicates if their Jaccard similarity

$\Rightarrow$ algorithmic challenge: find near duplicates in linear time $> \text{threshold}$

def MinHash(c):

A random hashfunction. h so that $P[h(A) = h(B)] = \text{Jaccard}(A, B)$

Normally, you want different items to hash to different

— but there, you want collision probability to depend on hashes, similarity

def: locality-sensitive hashing ()

Summary:

①: Algorithmic tools: n-gram models (KenLM), classifiers (fastText), importance resampling (DSIR)

②: Application = language identification, quality filtering, toxicity filtering

③: Deduplication: hashing scales to large datasets for fuzzy matching

Jaccard-similarity $\downarrow$ MinHash $\downarrow$ locality-sensitive hashing

④: Now you have the tools (mechanics), just to have to spend time with data (iterations)

Lecture #15: 详解 SFT + RLHF

↳ supervised-fine-tune

Reinforcement Learning
Human feedback

previous: cover ~~pre-train~~ pre-training: GPT3 remarkable model

to day: cover post-training: instruct-GPT: instruction

goal: enable better, tighter controls over LLM output

↳ SFT, followed by 'RL' HF.

input → Policy Model → Output → Reward Model → Reward → RL algorithm → ^{update} Policy

step #1: collect demonstration data, and train a supervised policy

step #2: collect comparison data, and train a reward model

step #3: optimize a policy against the reward model using

reinforcement learning

↳ RLHF policy model

modified
LLM

step #1

three classical instruction-tuning data:

(FLAN, Oasst, Alpaca - Stanford, Open-Assistant

①: length and bullet points (style variations)

②: Reference, richer knowledge, less invisible,

③: scale but important aspects.

④: safety: how do the factors affect the model?